SymBot-Full · 插件归属与记忆内核深度评测

被审对象：C:\AIGCProgram\GitHub\SymBot-Full（衡忆数字生命 0.1.0，variant=full，built_at 2026-11-02T23:14:45+0800） 评测日期：2026-11-03 标准：四项目评测标准 v1.0
模型：deepseek-ai/DeepSeek-V4.1-Flash 取证：.tmp/bench/plugin_map.md（306 行） + .tmp/bench/memory_evidence.md（386 行） + 本机实跑 848 项断言
纪律：全程只读被审仓库；未改源码、未改数据库、未联网抓取

第一部分 哪些能力是插件实现的

1.1 一句话

她的「人」100% 是核心自研，她的「嗓子」100% 是插件。

人格、记忆、关系、作息、群聊调度、面板、OpenAI 兼容入口，全是 app/ 里自己写的（131 文件 / 35,723 有效行）；

语音合成、换声翻唱、乃至 QQ 接入，全部依赖仓库外或 .third_party/ 里的第三方件。

全量版那个「9.4 GB」，装的就是这副嗓子。

1.2 三层边界

层	是什么	归属	在不在仓库里
核心	人格/记忆/关系/生活/面板/工具	自研	在（48,129 有效行）
插件	肥鱼 CosyVoice3 声音服务	第三方（AstrBot 插件形态）	在 .third_party/feiyu_voice/（9.1 GB）
宿主	NapCat（OneBot v11）	第三方	不在，用户自己装

1.3 那个插件到底是什么

不是「几个 py 文件」，是三样东西塞进一个 git 仓：

1. 一个正牌 AstrBot 插件 —— metadata.yaml:1-8 声明 astrbot_version >=4.29.3,<5，_conf_schema.json 是 AstrBot 的配置 Schema，main.py:14 是 @register 装饰器，:169/:184/:201 注册了 3 个 llm_tool。也就是说：这个包本来是给 AstrBot 写的插件，本项目拿去当外部服务用。
2. 一个独立 FastAPI 声音服务 —— server/feiyu_voice_service/app.py:84-188 暴露 /v1/speech(202)、/v1/voice-conversion、/v1/jobs/{id}/audio，默认 127.0.0.1:9880（__main__.py:18-19），Bearer 鉴权在 app.py:68-80。
3. 7,190.8 MB 权重 —— server/configs/voice_service.json:6-20：cosyvoice3_release（TTS） + feiyu_rvc_release（换声），歌唱位 singing: null（也就是说唱歌能力没启用，翻唱走的是另一条 demucs+RVC 管线）。

核心侧只做了一件事：写了客户端。app/voice/tts.py:3-5 明说「与『肥鱼声音 / astrbot_plugin_feiyu_voice』的服务端一致，本项目自己实现客户端」，契约就是 POST {base_url}/v1/speech {"voice","text","style"}。

1.4 体积与代码占比（真实数字）

指标	核心	.third_party	占比
文件数	2,975	13,724	插件 82.18%
字节	449,210,974	9,544,360,606	插件 95.51%
Python 有效行	48,129	1,296,910	插件 96.4%（含上游 CosyVoice/RVC 源码）

两句流行的说法，一句对一半、一句不成立：

- 「体积 97% 是插件」—— 方向对，数字要改成 95.51%。
- 「代码 100% 是核心」—— 不成立。插件侧的 Python 有效行是核心的 27 倍，只是绝大部分是上游模型源码。
- 准确的说法是本文开头那句：人 100% 核心，嗓子 100% 插件。

1.5 插件缺席时会发生什么（这一节最该看）

评测标准把「静默失败」列为重罪。这个项目同一份代码里有两种标准：

场景	表现	判定	证据
TTS 合成失败	静默降级成文字，无提示、无日志	重罪	app/api/onebot.py:1046-1047 except Exception: pass # TTS 失败就当没合成，文字照发；app/voice/tts.py:216-217 把 ConnectError 包成 {"ok":False} 不抛也不 log
记忆内核缺失	显式抛 MemoryCoreUnavailable	正确	app/memory/api.py:10-11 注释写明「绝不静默返回空结果」
翻唱失败	有回话，但原因统一说成「音太高」	失真	真因 rvc-error 在 app/voice/cover.py:593-596 就被丢掉；用户听到的是 app/api/onebot.py:178-179「这首我练了半天没练下来，可能音太高了……」
人声分离环境缺失	直接不可用	硬伤	app/voice/cover.py:61 _SEP_PY_DEFAULT = r"C:\feiyu\sep\Scripts\python.exe" —— 作者机器上的路径
一键启动	不拉语音服务	文档不符	一键启动.bat:10 只跑 app.cli serve；而 上手指南.md:119 写「Full 版：一键启动.bat 会顺手拉起」。真正会拉的是 app/core/services.py:571 start_voice()，入口在面板服务抽屉，不在 bat

这就是为什么评测口径 B 里它拿不到满分：四件事里有三件会静默或失真。

1.6 模块热插拔：系统在，生态是空的

app/core/modules.py 686 行 + app/api/modules_api.py 88 行，做得相当扎实：

- manifest 六个必填字段校验、id 必须等于目录名、entry 三道关（不许 ..、必须在模块目录内、必须 .py）、requires 只校验不代装；
- 能注册工具 / 路由 / 钩子三类，reload 时摘旧路由再插到面板 Mount("/") 之前；
- 注册到一半炸了会回滚，停用只是不 import（文件不动），开关存 kv；
- 面板三接口：列表 / 开关 / 重载。

但三个事实把它按在地上：

1. 全仓模块只有一个 hello，它的 manifest 自己写着「验收用」（modules/hello/manifest.json:9）。
2. register_tool 是死接线：模型工具表硬编码 3 个（app/tools/registry.py:16-35），规划器只认这 3 个（app/tools/router.py:46-69），全仓没有任何地方把 modules.registry().tools 喂给模型。
3. MCP 全仓 grep = 0。没有权限模型、没有配额、没有市场。

结论：插件系统是给未来留的口子，现在装不了任何真功能。 这一维给 4 分（从基线 3 分 +1），不是 7。

1.7 插件部分该批评的三件事

1. TTS 失败 except: pass，而记忆内核做对了 —— 同一份代码两套标准，说明这里不是不懂，是没顾上。
2. 翻唱把「服务没起 / 没 key / 显存不够」全部输出成「音太高」，用户永远修不好。

3 C:\feiyu\sep 这种作者机器路径直接进了发布版源码 —— 这不是配置项，是常量。

第二部分 记忆内核详细评测

2.1 五分钟看懂

```
用户说话
  └─ h2.add()                ← 唯一写入口（去重靠正文全等）
     ├─ mem_item             ← 真表（22 列 / 6 索引），记忆本体
     ├─ mem_cue              ← 线索表（n-gram，召回的唯一钥匙）
     └─ mem_event            ← append-only 档案，永不删改

用户问「你还记得吗」
  └─ h2.retrieve()
     ├─ 线索闸门：没有任何 cue 命中 → 直接出局（纯属性分不能进候选）
     ├─ 打分：cue 34/18 + importance 30 + heat 20 + clarity 15/10/4/0
     │        + pinned 25 + recency 12，阈值 55
     ├─ 热层：pinned 或 decayed_heat ≥ 0.50，最多 10 条（只读，不加温）
     └─ 刚想起来的：hit 取 2 条、uncertain 取 1 条，命中才 +0.28 热度

进 prompt：`【你记得的】`（每类 ≤8 条）+ `【之前聊过】`摘要 + `【之前没说完的】`
```

2.2 规模

项	数字
app/memory/*.py	24 个，5,764 行 / 216,615 字节
最大三个	h2.py 578 行、reference.py 293、memv2.py 247
闭源容器	_vault.bin 154,012 字节，38 成员（37 源文件 + 清单）
容器内最大	db/repo.py 92,968 字节 / 2,428 行（全部 SQL 的唯一出口）、recall.py 37,103 字节
引擎版本	__version__ = "0.3.0"，顶层公开 API 62 个

2.3 存储：memories 是视图，不是表

这是整套设计的地基，也是它的第一条不变量：

```
CREATE VIEW memories AS
SELECT m.id, m.text AS content, m.importance/100.0 AS importance, ...
FROM mem_item m
WHERE m.forgotten = 0 AND m.superseded_by IS NULL AND m.merged_into IS NULL
```

（app/memory/memv2.py:32-58）

- 真表 mem_item 22 列 6 索引；线索在 mem_cue（ON DELETE CASCADE）；档案在 mem_event，注释写明 append-only。
- 全仓 CREATE TRIGGER 命中 0 —— 「记忆写入只走 h2.add()」这条规则靠代码纪律，不靠数据库约束。
- 好处：视图天然过滤（遗忘/被取代/被合并三种条目自动不可见）。
- 代价：没有级联。系统从不物理删 mem_item，所以 ON DELETE CASCADE 永远不触发，线索被永久留下。

2.4 写入：一个入口，一条纪律

h2.add() 是唯一入口，memories 视图拒绝写入（回归 D1 专门测这条）。去重是：

同人 + 同场合 + 同 kind + 正文完全相等 → 判定重复，max(heat, 0.90) + 0.1 加温（app/memory/h2.py:249-254）。

改一个字就是新记忆。语义去重 = 0（全仓「语义去重」只在 docs/00-overview 出现一次，且是自认「没有」）。

2.5 召回：字符串匹配 + 16 个手调权重

这是整套记忆最被高估、也最值得看清的部分。

闸门：第一轮里，如果一条记忆「没有任何线索命中且没有实体命中」，直接返回 None —— 纯属性分不能进候选（recall.py:788-789）。

换句话说：你问的话里没有它身上的词，它就永远想不起来，哪怕它重要度 100、热度 1.0、被钉住。

打分（16 个常量都在 RecallWeights，recall.py:71-88）

项	公式	满分贡献
精确线索命中	34.0 × min(2.0, cue.weight)	34
包含命中	18.0 × min(2.0, cue.weight)	18
线索总分封顶	min(线暴分, 60)，实体 40×2 —— 合并封顶 140	140
重要度	importance/100 × 30.0	30
热度	clamp(stored_heat,0,1) × 20.0（用库里的原值，不是 decayed）	20
清晰度	vivid 15 / clear 10 / fuzzy 4 / trace 0（直接加，不乘权重）	15
钉住	+25.0 固定	25
新鲜度	max(0, 1 - age_days/30) × 12.0	12
阈值	hit_threshold = 55.0	—
置信度	confidence = clamp(top_score/100, 0, 1)	—

理论上界 242 分。也就是说：一条中等线索命中 + 一条 clear 记忆，就稳定过 55 分门槛 —— 门槛偏低。

confidence 不是概率，没有校准（calibrat 全仓 0 命中），它就是「最高分 ÷ 100」。

实测（本机真库副本，prod 模式）

场景	状态	confidence	cues	热层	刚想起	memory 段 tokens
chat「今天有点累」	hit	0.94	6	10	0	56
group「昨天那个游戏」	hit	—	6	10	1	61
recall「我妈住院的事」	hit	0.95	6	10	2	51 + 18
tool「汇率」	hit	0.94	6	10	1	59

召回确实在工作，也确实进了 prompt ——
这一条是真的，不是话术。但「hit」的含义是「有一条线索撞上且总分过了 55」，不是「她真的想起了那件事」。

有没有向量？没有。 app/ + scripts/ 169 个 py 里 vector / embedder / cosine / faiss / chroma / qdrant 全部命中 0；引擎侧写了 mem_embedding 表和可选 query_vector 通路，但宿主从不生成向量，那张表 0 行。所以「换个说法就记不起来」是结构性的，不是配置问题。

2.6 遗忘：线性 0.04/天，25 天归零

参数	值	含义
初始热度	0.90	新记忆起点
衰减	0.04/天，线性	1.0 → 0 恰好 25 天
命中加温	+0.28	一次命中 ≈ 续命 7 天
热层门槛	0.50	每轮进上下文的线
冷层门槛	0.12	要靠线索才捞得动

不是艾宾浩斯，不是指数，是线性。 而且宿主的 h2.py:41 里那个 DECAY_PER_DAY = 0.97 是死常量（注释自己写「旧口径，只留在文档里」），谁按 0.97 指数去理解衰减都会算错 ——

docs/06-memory.md:274 也自认了这一点。

线性衰减 + 25
天归零，对一个「长期记忆」定位的产品来说偏激进：一条没被cue到的日常小事，三周多就凉了。真正兜住的是 importance ≥ 60（豁免遗忘 + 清晰度最低只到 fuzzy）和 pinned。

引擎级遗忘判定（5 个 AND）：非 pinned、未遗忘、年龄 ≥ 60 天、清晰度 = trace、heat ≤ 0.10、importance < 40（decay.py:175-195）。

2.7 冷归档：合并可还原，但后悔药只能吃一次

- 准入：年龄 ≥ 30 天、heat < 0.12、非 pinned、kind 不在 boundary/promise/quote；
- 成组：同月 + 同 kind + 同一个人 + 同一个场合，凑够 3 条；
- 产物：一条 kind="event"、clarity="fuzzy" 的新记忆，heat 取组内最大值；
- 源条目：只打 merged_into = 新 id，行不删（软合并）；
- 还原：restore(merge_id) 把 merged_into 清空 → 热度/召回次数/线索全都不丢（因为这些列从没被改过）；
- 但：还原末尾 DELETE FROM memory_merges WHERE id = ?（cold.py:214），账本行被物理删掉 —— 同一个 merge 无法二次还原。

这是全套系统里最漂亮也最可惜的一块：可逆做得对，账本却只留一次。

2.8 压缩与摘要

- 触发：未压原文条数 > verbatim_max，或 token 超预算（两条件取或）；
- 保留：最近 keep_recent 轮永不压缩；
- 摘要不是事实：system prompt
  明令「不要写他的偏好、他的承诺、具体经历——那些另存」（compress.py:124-129）；
- 兜底摘要上限：话题 5 条、未完事项 3 条、总长 420 字；
- 只保留最近 5 条摘要，更旧的物理删除；
- 摘要进 prompt 的【之前聊过】，与【你记得的】互不替代（prompt.py:378-381）。

2.9 派生记忆层：6 个模块，只有 1 个真写记忆

模块	干什么	写哪里
facts.py	每 6 条用户消息抽一次，一次 ≤2 条事实	mem_item（唯一真写记忆的）
threads.py	抓她自己说的「明天/回头/改天」建提醒	open_threads 独立表，48h 去重
anchors.py	从语料重建常驻声纹/场景句	anchors 表（先 DELETE 再灌）
reference.py	语料按月分期（热恋/稳定/降温/破裂）	ref_turns / ref_quotes，明令「绝不进 memories」
declare.py	纯解析器，判断谁是我/谁是她	零 DB 写入
evolve.py	每日思考报告 + 每人印象 + 每周自评	human_notes / member_impression / reflect_report

设计上很清楚：进入「记忆本体」的只有三路 ——
用户原话（ingest）、模型抽的事实（facts）、关系事件（announce/veto）/ 冷合并摘要。 其余都是旁路表。

2.10 可编辑 / 可遗忘：能删能钉，不能改字

能力	有	证据
列表 / 按人分组	有	GET /v1/memories（routes.py:181-208）
钉住	有	POST /v1/memories/{mid}/pin（:269-274），pinned=True → heat=1.0；取消钉住回落 0.90（不回原值）
删除单条	有	DELETE /v1/memories/{mid}（:284-288），软删 forgotten=1

能力	有	证据
清空某人 / 全库	有	forget_user / forget_all，全清走 /v1/dev/memory/clear
冷压缩 / 还原	有	/v1/dev/memory/{compact,cold,restore}
改一个字	无	routes.py 里 @router.put / @router.patch 命中 0

这是评测维度里「可编辑」一项的真实答案：只能增、删、钉、并，不能改。
写错一个字，只能删了重记（而且重记得等下一次对话）。

删除是软删，mem_event 永不删 —— 所以「忘记」=
从视图消失，物理数据仍可导出/审计，不是不可恢复。这对隐私是好是坏，看你要哪个。

2.11 记忆在 prompt 里只占 1.8%

这是最该被记住的一个数。本机实测（budget.sections，内核自报）：

段	tokens	占比
语感 speech	687	21.6%
人格卡 identity	447	14.1%
状态提示 state_hint	233	7.3%
底部 bottom / 沉浸 immersion	148 + 83	7.3%
历史对话（差值）	~1,101	34.6%
禁区 forbid / 思考 think / 关系 relation / 日记 diary / 摘要	86+121+105+40+37	12.2%
记忆 memory	56	1.8%
召回 recalled	0（recall 场景 18）	0（1.3%）

一套 5,764 行、24 个模块的记忆系统，在这轮对话里只占 56 个 token。

不是被截断（used_est 3,180 / 窗口 64,000，只用了 5%），是设计上就给了这么点：热层 10 条 + 刚想起 2 条，【你记得的】每类上限 8 条。

结论：记忆在她的「人格构成」里是配菜，不是主菜。 语感（687）+ 人格（447）是主菜。

2.12 「闭源」是包装，不是壁垒

项	实测
容器格式	59 字节头 + 密文 + 32 字节 tag
密钥派生	PBKDF2-HMAC-SHA256(machine_secret, salt, 120000) → 分 enc/mac 两把
加密	HMAC-SHA256 计数器流 XOR，纯标准库
完整性	有 MAC，compare_digest 先验后解
MAC 密钥	就是本机指纹（platform.system/machine + 主机名 + MAC48）
清单校验	_vault_manifest.json 附在容器里，运行时不校验（_loader.ensure_core() 直接 extractall）
同机可解开?	可以。取证时已在内存里完整解出 37 个源文件

所以「防篡改」只防别的机器，不防本机用户。换机/改计算机名/换网卡会解不开（mode=0 机器绑定），搬运要靠 SYM_MEMORY_VAULT_PASSPHRASE 口令模式。

这件事对评测的影响是双向的：

- 对用户：不是「黑盒不可审计」，只是「装了层壳」，出问题能查；
- 对宣称：「闭源记忆」这四个字站不住 —— 它是混淆 + 本机绑定，不是密码学壁垒。

另外，app/memory/api.py:118-164 是双路径回退：先试 vendor/ 源码，失败再解容器。发布包里 vendor/ 已经删了，所以默认走容器；但只要把源码放回去，源码优先。

2.13 测试覆盖与盲区

套件	总断言	记忆相关
scripts/smoke.py（2,378 行）	390	22 条（写入/隔离/私聊群聊分流/视图拒写/钉住/一次召回一条审计/冷合并可从视图消失/还原/清记忆留痕）
scripts/fullcheck.py（705 行）	153	7 条路由（recall 结构、导出含引擎归档、口令 403、cold/compact/restore、console memory）
scripts/test_intimacy.py	60	6 条（遗忘后关系归零/事件清空/记忆清空/不影响乙）
其余四套	210	0

做得好：视图拒写、按人隔离、私聊群聊分流、一次召回只落一条审计 —— 这几条是真正的边界，全被钉住了。

盲区（代码里找不到断言）：衰减曲线的数值、RecallWeights
每个常量的边界、并发写入、容器被替换后的行为、restore() 二次调用。

2.14 三个死代码 + 八个缺口

死代码（最该修）

1. h2.py:41 DECAY_PER_DAY = 0.97 —— 永远不参与计算，但长得像真相。
2. add() 的 stability 参数 —— 函数体零引用，mem_item 没这列，to_old() 恒定输出 0.5。
3. 关系加成是死的 —— scoring.py:35 有 RELATION_BONUS = {0:0,1:10,2:20,3:30}，但宿主从不写 meta["relation_stage"]（全仓 grep 命中 0），所以永远回落到 STRANGER。「关系越亲密记忆越牢」实际只有 importance ≥ 60 这一条硬阈值在工作。

八个缺口

#	缺口	后果
1	无向量检索落地	换个说法就记不起来
2	语义去重 0（正文全等才算重）	同一件事存两条
3	置信度不校准（分数÷100）	0.95 不代表 95% 会命中
4	多模态记忆 0	图/语音结果不进 mem_item
5	无跨用户共享机制	只能拿 user_id="0" 冒充公共区
6	无物理删除、无按 kind 的 TTL	「忘记」≠ 真的没了
7	无编辑接口	正文写错只能删
8	关系阶段加成死代码	宣传语里的能力没接线

2.15 分轴打分

评测标准里 memory 是一个 0–10 的整体分。为了看清它，这里把它拆成五轴：

轴	分	依据
结构完整度	10	视图+三表、唯一写入口、按人分区、热/冷双层、软合并可还原 —— 别家没有第二家做全
召回质量	6	纯字符串+16 权重，门槛 55 偏低，无向量无语义；但线索闸门与共线索扩散设计得对
遗忘设计	7	线性 0.04/天、25 天归零偏激进；importance≥60 与 pinned 兜底算合理
可维护性	6	可解释、可审计、有 22 条断言；但三个死代码 + 容器层混淆 + 无编辑接口
提示词占比	4	记忆只占 system 的 1.8%，是配菜不是主菜

综合 9 分（评测标准里的整体分）：打的是结构完整度与工程纪律，不是召回准确率。

如果用「召回准确率」当尺，同源代码只能给 6–7 分 —— 这句话必须写清楚，否则 9 分会被当成「语义记忆」。

2.16 与另外四家比记忆

项	SymBot-Full	SMD-P	surfcore	qq-bridge	AstrBot
按人分区	有 user_id	有	有 角色卡	弱 会话级	无
热度/衰减	有 线性 0.04/天	有 同源	无	无	无
冷归档可还原	有 软合并+restore	有 同源	无	无	无
可遗忘	有 软删+留痕	有	弱	弱	无
可编辑正文	无	无	有 卡片	有 文件	无
语义/向量召回	无	无	弱 embedding	无	无
记忆相关测试	22+6 条	22+6 条	少量	无	无
整体分	9	9	7	5	3（missing）

一句话：记忆这件事上，它是五家里唯一把「写、记、忘、归档、还原」做全的；也是五家里唯一一个「召回全靠字符串」的高分方案。 前者是真本事，后者是真短板。

第三部分 结论

1. 插件归属很清楚：人格/记忆/关系/生活/面板是你的，语音/换声/翻唱是人家的，QQ 是 NapCat 的。全量版 95.51% 的体积买的是嗓子。
2. 插件系统存在但没有生态：686 行运行时、面板三接口、自测两套都到位；register_tool 没接进模型，模块只有 hello 一个示例。这一维从 3 分提到 4 分，不能再高。
3. 记忆是全场最强，但强在结构不在算法：视图+三表、唯一写入口、按人分区、软合并可还原、22+6 条断言，这些是真护城河；无向量、无语义去重、置信度不校准、关系加成是死代码、记忆只占 prompt 1.8%，这些是真短板。
4. 最该先修的三件事：① TTS 的 except: pass 改成显式提示；② 删掉/接上 DECAY_PER_DAY、stability、RELATION_BONUS 三个死代码；③ 补一个 PATCH /v1/memories/{mid}。

评分口径：本文的第二部分是取证与分轴，不改变评测标准里 memory=9 的整体分；两者口径已在 2.15 说明。